Chapter 9

Text and regular expressions

describe the shape of the text; the engine finds it

String methods (`split`, `strip`, `replace`, `startswith`, `in`) handle fixed text. When what you're looking for has a **shape** rather than a fixed value ("a date", "an email", "a word followed by a number"), you describe the shape with a **regular expression** and the `re` module finds every piece of text that fits:

```
re.finditer(r'\d{4}-\d{2}-\d{2}', text)                    3 matches
Loans: Dune due 2026-10-03, Emma due 2026-11-15 (renewed 2026-10-20).
```

Each box is one match. In this chapter every pattern is drawn like this, so you can **see** what it matches before trusting it. Capturing groups get coloured underlines:

```
re.finditer(r'(\w+) due (\d{4})-(\d{2})', text)            2 matches
Loans: Dune due 2026-10-03, Emma due 2026-11-15.
```

RULE OF THUMB

Always write patterns as raw strings: `r"\d+"`. Without the `r`, Python processes backslashes first (`"\b"` is a backspace character), and your pattern silently changes.

HONEST NOTE

Regular expressions are powerful and hard to read. If `str` methods can do the job, use them. Never parse HTML, JSON or a programming language with a regex: use a real parser (`html.parser`, `json`, `ast`).

Formatting: the mini-language after the colon

Inside an f-string, everything after `:` is a **format spec**: `{value:fill align width , .precision type}`. It's the same mini-language used by `format()` and `str.format`:

```python
title, price, count, ratio = "Dune", 7.5, 1234567, 0.4567

print(f"[{title:<10}] [{title:>10}] [{title:^10}] [{title:*^10}]")
print(f"[{price:8.2f}] [{count:,}] [{count:_d}] [{ratio:.1%}] [{count:.2e}]")
print(f"[{42:05d}] [{255:x}] [{255:#b}] [{-3:+d}] [{3:+d}]")
print(f"{title=}, {price * 2 = }")                    # self-documenting, for debugging
width = 12
print(f"[{title:>{width}}]")                           # nested: width from a variable
```

```
[Dune      ] [      Dune] [   Dune   ] [***Dune***]
[   7.50] [1,234,567] [1_234_567] [45.7%] [1.23e+06]
[00042] [ff] [0b11111111] [-3] [+3]
title='Dune', price * 2 = 15.0
[        Dune]
```

spec	means	example → result
`< > ^`	align left / right / centre in the width	`f"{'ab':>4}"` → `'  ab'`
`*^10`	fill character before the alignment	`f"{'ab':*^6}"` → `'**ab**'`
`.2f`, `.3g`, `.1%`, `.2e`	fixed decimals, significant figures, percent, exponent	`f"{0.5:.0%}"` → `'50%'`
`,` or `_`	thousands separator	`f"{10**6:,}"` → `'1,000,000'`
`05d`, `x`, `b`, `#x`	zero-pad, hex, binary, with prefix	`f"{10:#x}"` → `'0xa'`
`!r`	use `repr()` instead of `str()`	`f"{'a'!r}"` → `"'a'"`

Unicode: when equal-looking strings aren't equal

"é" can be stored as one code point (U+00E9) or as "e" plus a combining accent (U+0065 U+0301). They look identical and compare unequal. **Normalise** text from outside before comparing, and use `casefold()` (not `lower()`) for case-insensitive matching:

```
>>> import unicodedata
>>> a, b = "caf\u00e9", "cafe\u0301"
>>> a == b, len(a), len(b)
(False, 4, 5)
>>> unicodedata.normalize("NFC", a) == unicodedata.normalize("NFC", b)
True
>>> "Straße".lower() == "STRASSE".lower(), "Straße".casefold() == "STRASSE".casefold()
(False, True)
>>> "".join(c for c in unicodedata.normalize("NFKD", "Zoë Ångström") if not unicodeda…
'Zoe Angstrom'
```

Regex building blocks

A pattern is built from a small set of pieces. Learn these and you can read most regexes:

piece	matches	piece	matches
`abc`	exactly "abc"	`.`	any character except newline
`\d \w \s`	digit, word char (letter, digit, _), whitespace	`\D \W \S`	the opposite of each
`[aeiou]`	one of these	`[^0-9]`	one character NOT in the set
`x* x+ x?`	0 or more, 1 or more, 0 or 1	`x{3} x{2,4}`	exactly 3, between 2 and 4
`^ $`	start / end of the string (or line)	`\b`	word boundary (between \w and \W)
`a|b`	a or b	`(...)`	group: capture, and apply `*`, `|` to the whole
`(?P<name>...)`	named group	`(?:...)`	group without capturing

Build a pattern up one piece at a time, checking each step against real text:

`re.finditer(r'\d+', text)` — **9 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'\d{5} \d{6}', text)` — **1 match**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'(\+44 |0)7\d{3} \d{6}', text)` — **2 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'(\+44 |0)(7\d{3}) (\d{6})', text)` — **2 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

WHY

Step 1 matched every run of digits, including the fax and the reference. Step 2 required the exact shape but missed the +44 form. Step 3 made the prefix an alternative and fixed the leading 7. Step 4 added groups to pull out the parts. That loop (match too much, tighten, check) is how regexes are written in practice.

Using re: search, findall, groups, sub

```python
import re

line = "2026-09-28 14:03:11 WARN loans: member=ana book=Dune days_over=3"
m = re.search(r"(?P<date>\S+) (?P<time>\S+) (?P<level>[A-Z]+) (?P<area>\w+):", line)
print(m["level"], m.group("area"), m.span(), m.groupdict()["date"])

pairs = dict(re.findall(r"(\w+)=(\w+)", line))           # with 2 groups: list of tuples
print(pairs)

print(re.fullmatch(r"[A-Z]{2}\d{2} ?\d[A-Z]{2}", "M13 9PL"))   # whole string must match
print(re.split(r"[;,]\s*", "Dune; Emma,Ubik ,  Kim"))
```

```
WARN loans (0, 31) 2026-09-28
{'member': 'ana', 'book': 'Dune', 'days_over': '3'}
None
['Dune', 'Emma', 'Ubik ', 'Kim']
```

function	returns	use when
`re.search(p, s)`	first `Match` anywhere, or `None`	"is it in there, and where?"
`re.match(p, s)`	`Match` only at the start, or `None`	rarely; prefer `fullmatch` or `^`
`re.fullmatch(p, s)`	`Match` if the **whole** string fits	validating input
`re.findall(p, s)`	list of strings (or tuples of groups)	extracting every occurrence
`re.finditer(p, s)`	iterator of `Match` objects	you need positions, or big text
`re.sub(p, repl, s)`	new string with matches replaced	cleaning, redacting, rewriting

sub with a function

`repl` can be a function: it receives each `Match` and returns the replacement. That turns `sub` into a general text transformer:

```python
import re

def mask(m):
    user, domain = m["user"], m["domain"]
    return f"{user[0]}{'*' * (len(user) - 1)}@{domain}"

text = "Contact ana.reader@uni.ac.uk or ben@library.org for keys."
print(re.sub(r"(?P<user>[\w.+-]+)@(?P<domain>[\w-]+(\.[\w-]+)+)", mask, text))
print(re.sub(r"(\d+) days", lambda m: f"{int(m[1]) * 7} days", "renew for 2 days, then 3 days"))
```

```
Contact a*********@uni.ac.uk or b**@library.org for keys.
renew for 14 days, then 21 days
```

Greedy, lazy, and readable patterns

`*` and `+` are **greedy**: they grab as much as possible and only give back what they must. Adding `?` makes them **lazy** (as little as possible). The classic surprise:

`re.finditer(r'<.*>', text)` — **1 match**

Dune and Emma

`re.finditer(r'<.*?>', text)` — **4 matches**

 Dune and Emma

`re.finditer(r'<[^>]*>', text)` — **4 matches**

 Dune and Emma

Greedy `.*` ran to the **last** `>`, swallowing the whole line. The lazy version stops at the first `>`. A negated class `[^>]*` ("anything but >") says exactly what you mean and is usually fastest.

Long patterns become readable with `re.VERBOSE`: whitespace is ignored and `#` starts a comment:

```python
import re

POSTCODE = re.compile(r"""
    ^ (?P<area> [A-Z]{1,2} )      # 1-2 letters: M, SW
      (?P<district> \d[A-Z\d]? ) # digit, optional digit/letter: 13, 1A
    \s?                           # optional space
      (?P<sector> \d )            # one digit
      (?P<unit> [A-Z]{2} ) $      # two letters
""", re.VERBOSE | re.IGNORECASE)

for pc in ["M13 9PL", "sw1a 1aa", "M13-9PL"]:
    m = POSTCODE.match(pc)
    print(pc, "->", m and m.group("area", "sector"))
```

```
M13 9PL -> ('M', '9')
sw1a 1aa -> ('sw', '1')
M13-9PL -> None
```

TRAP

Some patterns take exponential time on certain inputs (catastrophic backtracking), e.g. `(a+)+$` against `"aaaaaaaaaaaaaaaaaaaaaaaa!"`. If a regex runs on text from users, keep it simple, avoid nested quantifiers, and limit the input length. This has taken down real websites.

A scanner: turning text into tokens

Compilers, calculators and config parsers all start by splitting text into **tokens** (Ch 2). One regex with a named alternative per token type, scanned left to right, does it in a few lines:

NUMBER (orange) · NAME (purple) · OP (green) **11 matches**

total = price * (1 + rate_2) - 3.5

```python
import re

TOKEN = re.compile(r"""
    (?P<NUMBER> \d+(?:\.\d+)? ) | (?P<NAME> [A-Za-z_]\w* ) |
    (?P<OP> [-+*/=()] )         | (?P<SKIP> \s+ )           | (?P<BAD> . )
""", re.VERBOSE)

def tokens(text):
    for m in TOKEN.finditer(text):
        kind = m.lastgroup                    # the name of the group that matched
        if kind == "SKIP":
            continue
        if kind == "BAD":
            raise SyntaxError(f"unexpected {m.group()!r} at column {m.start()}")
        yield kind, m.group()

print(" ".join(f"{kind}:{text}" for kind, text in tokens("total = price * (1 + rate_2)")))
print(list(tokens("2 $ 3")))
```

```
NAME:total OP:= NAME:price OP:* OP:( NUMBER:1 OP:+ NAME:rate_2 OP:)
SyntaxError: unexpected '$' at column 2
```

Other text tools in the standard library

```python
import string, textwrap

t = string.Template("Dear $name, '$title' is due on ${day}.")
print(t.substitute(name="Ana", title="Dune", day="Friday"))
print(textwrap.fill("Regular expressions are a small language for describing text.", width=28))
print(textwrap.shorten("A very long book title that will not fit", width=24, placeholder="..."))
```

```
Dear Ana, 'Dune' is due on Friday.
Regular expressions are a
small language for
describing text.
A very long book...
```

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 9.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 9.1` shows other valid solutions and their trade-offs.

`9.1` build from a spec URL slugs from any title
`9.2` build from a spec Parse a log line with named groups
`9.3` fix the bug The greedy tag stripper
`9.4` build from a spec A text table with format specs
`9.5` build from a spec Redact personal data
`9.6` mini-project Mini-project: a calculator from scratch

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

`str` is immutable Unicode text (code points); normalise with `unicodedata`, compare with `casefold`. Format specs: `{x:>10,.2f}`. Regex via `re` (Perl-style syntax, raw strings, named groups, `sub` with functions, `VERBOSE`).

12. Stdlib and tooling

`re`, `string`, `textwrap`, `unicodedata`, `difflib` cover most text work.

IN OTHER LANGUAGES

Regex syntax is nearly the same everywhere (Perl-compatible): JavaScript `/\d+/g`, Java `Pattern.compile`, `grep -E`, your editor's search box. What changes is the API around it and a few features (named group syntax, lookbehind support).

DEEP DIVE (your vault)

Vault notes: 5.11 Strings, Unicode, and textual value · 5.12 Bytes and the text boundary · 7.9 Strings as immutable sequences with textual semantics.